



2024

开发者测试概述



开发者测试概述



早期软件规模小且复杂程度低，软件测试常常包含于调试工作中，由**软件开发人员**完成相关工作



在当今规模化和工程化的软件研发中，为了提高生产效率和专业化程度，逐步产生了岗位细分，出现了**专门的软件测试岗位**



当前移动互联网迅速普及的背景下，众多软件公司纷纷采用了微小改进、快速迭代、反馈收集、及时响应等手段来迅速改进软件产品，满足用户需求



软件的持续快速迭代需求，大大压缩了软件开发的发布流程，使得一部分测试任务开始迁移，由软件开发人员担任这部分跟代码相关的软件测试工作，称为**开发者测试**

目录

CONTENTS

PART 01 开发者与软件测试

测试和调试

开发者测试

PIE模型

PART 02 开发者测试方法与技术

静态测试与动态测试

黑盒测试与白盒测试

PART 03 开发者测试趋势

软件开发困境

DevOps介绍

DevOps中开发者测试

A stylized graphic of a book with the number 01. The number 01 is in a bold, dark blue font. The book is depicted with a white cover and a dark spine, with a small URL 'ibaotu.com' visible on the spine. The book is positioned behind the number 01.

01

开发者与软件测试

目录

CONTENTS

PART 01 开发者与软件测试

测试和调试

开发者测试

PIE模型

PART 02 开发者测试方法与技术

静态测试与动态测试

黑盒测试与白盒测试

PART 03 开发者测试趋势

软件开发困境

DevOps介绍

DevOps中开发者测试



测试和调试



图 1-1 调试的过程



软件开发过程中，开发者需要对程序进行测试和调试。测试和调试两者极其相关但含义完全不同。简单来说，测试是为了发现Bug（缺陷），调试是为了修复Bug

测试和调试



软件开发调试有时难度很大，原因如下：

- （1）失效症状和缺陷原因可能相隔很远，高度耦合的程序结构加深了这种情况；
- （2）缺陷症状可能在另一缺陷修复后消失或暂时性消失；
- （3）失效症状由不太容易跟踪的人为错误引发的；
- （4）失效症状可能时由不同原因耦合引发的。



程序调试方法多种多样，更多的时候是依赖程序员的经验和对程序本身的理解

目录

CONTENTS

PART 01 开发者与软件测试

测试和调试

开发者测试

PIE模型



PART 02 开发者测试方法与技术

静态测试与动态测试

黑盒测试与白盒测试

PART 03 开发者测试趋势

软件开发困境

DevOps介绍

DevOps中开发者测试



开发者测试



从履行职责、提高效率、保护源码、方便实现等角度来说，开发者需要完成的测试工作，主要集中在单元测试和集成测试阶段



静态测试与动态测试都是开发者需要掌握的测试方法，并在实践中两者结合使用



白盒测试是开发者最主要的测试方法，也是在软件测试工作上体现开发者优势的地方



开发者测试中，手工测试与自动化测试都会用到



开发者测试



为提高软件质量，缩短软件项目总的工期，测试常常和开发同步进行

ibaidu.com



同时，研发人员应综合运用多种软件测试方法和技术，并针对不同的测试场景应当合理选择测试方法和工具



此外，在测试时尽可能采用自动化测试工具来提高软件测试的效率

目录

CONTENTS

PART 01 开发者与软件测试

测试和调试

开发者测试

PIE模型



PART 02 开发者测试方法与技术

静态测试与动态测试

黑盒测试与白盒测试

PART 03 开发者测试趋势

软件开发困境

DevOps介绍

DevOps中开发者测试

PIE模型



在介绍PIE模型之前，我们首先理解Bug在不同阶段的不同名称及其含义，分别称为Fault、Error和Failure：

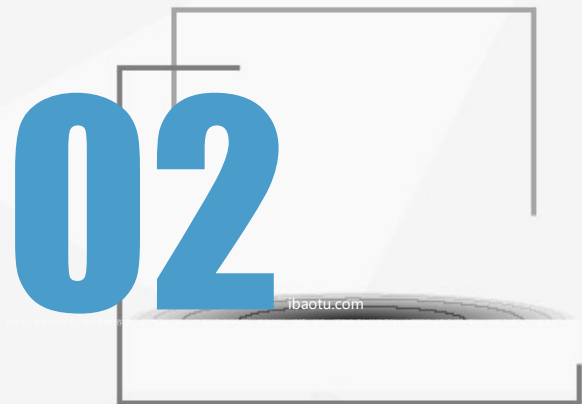
- ① **Fault**（故障）：故障是指静态存在于程序中的缺陷代码，有时也称之为程序缺陷（Defect）
- ② **Error**（错误）：错误是指程序运行缺陷代码后导致的错误状态
- ③ **Failure**（失效）：失效是指程序错误状态传播到外部被感知的现象

PIE模型



要发现一个Bug，需要满足以下三个必要条件：

- ① Execution-运行：测试必须运行到包含缺陷的程序代码
- ② Infection-感染：程序必须被感染出一个错误的中间状态
- ③ Propagation-传播：错误的中间状态必须传播到外部被观察到

A stylized graphic of a book with the number 02. The number 02 is in a large, bold, blue font. The book is depicted with a white cover and a dark spine, with a thin black line representing the text on the page. The background is a light gray with a subtle geometric pattern of overlapping triangles.

02

开发者测试方法与技术



开发者测试分类



软件测试从是否需要运行程序可以分为静态测试与动态测试；从是否需要了解软件内部结构可以分为黑盒测试和白盒测试

目录

CONTENTS

PART 01 开发者与软件测试

测试和调试

开发者测试

PIE模型

PART 02 开发者测试方法与技术

静态测试与动态测试

黑盒测试与白盒测试

PART 03 开发者测试趋势

软件开发困境

DevOps介绍

DevOps中开发者测试

静态测试



静态测试不运行被测程序，而是手工或者借助专用的软件测试工具来检查软件文档或程序是否符合标准、度量程序静态信息、审查软件中的问题和不足，降低软件缺陷出现概率



针对源程序的静态测试主要包括代码评审、代码结构分析、代码质量度量等



代码评审通常在动态测试之前进行。在审查前，应准备好需求描述文档、程序设计文档、源代码清单、代码编码标准和代码缺陷检查表等。代码评审在实践中根据检查流程和操作细节不同，可以细分为代码走查、桌面检查、代码审查等。代码评审主要检查代码与设计的一致性，代码对标准的遵循、代码可读性、代码逻辑表达的正确性，代码结构的合理性等方面。代码评审项目包括变量检查、命名和类型审查、程序逻辑审查和程序结构检查等

动态测试



动态测试是指通过运行被测程序，输入测试数据，检查运行结果与预期结果是否相符来检验被测程序功能是否正确



测试用例是动态测试中的关键所在。测试用例至少要包含两方面内容：测试输入数据和测试预期输出



在实际应用中，一个测试用例还可能包括测试环境、测试步骤、测试脚本、历史关联等信息

目录

CONTENTS

PART 01 开发者与软件测试

测试和调试

开发者测试

PIE模型

PART 02 开发者测试方法与技术

静态测试与动态测试

黑盒测试与白盒测试



PART 03 开发者测试趋势

软件开发困境

DevOps介绍

DevOps中开发者测试



黑盒测试



黑盒测试是不需要了解软件内部结构的测试方法的统称。待测程序被看作一个黑盒子，不考虑程序的内部结构和特性，测试者只知道该程序输入和输出之间的关系，依靠这些关系确定测试用例，然后运行程序，检查输出结果的正确性



最常用的黑盒测试方法是等价类测试和边界值测试，同学们可以参阅其他测试教材或参考书



白盒测试



白盒测试是需要了解软件内部结构的测试方法的统称。它把待测程序看成是一个可以透视的盒子，能看清楚盒子内部的结构以及是如何运作的

ibaidu.com



白盒测试依赖于对程序内部结构的分析，针对特定条件或要求设计测试用例，对软件的逻辑路径进行测试

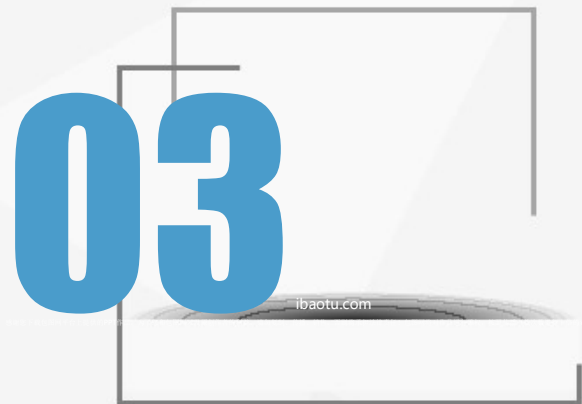
总结



黑盒测试多是动态测试，因为黑盒测试一般需要运行被测试程序。白盒测试既有静态测试，如代码评审、静态结构分析等，也有动态测试，如逻辑覆盖测试等。动态测试有可能是黑盒测试，如根据软件规格说明书进行功能测试，也有可能是白盒测试，如针对源程序做逻辑覆盖测试



介于黑盒测试与白盒测试之间还有一种方法称之为灰盒测试。灰盒测试是指只有部分程序代码信息的测试方法



03

开发者测试趋势

目录

CONTENTS

PART 01 开发者与软件测试

测试和调试

开发者测试

PIE模型

PART 02 开发者测试方法与技术

静态测试与动态测试

黑盒测试与白盒测试

PART 03 开发者测试趋势

软件开发困境

DevOps介绍

DevOps中开发者测试





软件开发困境



很多组织将开发和系统管理划分成不同的部门。开发部门的驱动力通常是“频繁交付新特性”，而运营部门则更关注IT服务的可靠性和IT成本投入的效率。两者目标的不匹配，就在开发与运营部门之间造成了鸿沟，从而减慢了IT交付业务价值的速度

ibaozu.com



开发人员经常不考虑自己写的代码会对运营造成什么影响



开发人员倾向于使用有利于快速开发的工具：对代码修改更快的反馈、更低的内存消耗等



开发是由功能性需求（通常与业务需求直接相关）驱动的，运营是由非功能性需求（例如可获得性、可靠性、性能等）驱动的



软件开发困境



一般而言，当企业希望将原本笨重的开发与运营之间的工作移交过程变得流畅无碍，他们通常会遇到以下三类问题：

- ① 发布管理问题：很多企业有发布管理问题
- ② 发布/部署协调问题：有发布/部署协调问题的团队需要关注发布/部署过程中的运行
- ③ 发布/部署自动化问题：这些企业通常有一些自动化工具，但他们还需要以更灵活的方式来管理和驱动自动化工作--不必要将所有手工操作都在命令行中加以自动化

目录

CONTENTS

PART 01 开发者与软件测试

测试和调试

开发者测试

PIE模型

PART 02 开发者测试方法与技术

静态测试与动态测试

黑盒测试与白盒测试

PART 03 开发者测试趋势

软件开发困境

DevOps介绍

DevOps中开发者测试





DevOps介绍



在很多企业中，应用程序发布是一项涉及多个团队、压力很大、风险很高的活动。然而在具备**DevOps**能力的组织中，应用程序发布的风险很低。与传统开发方法那种大规模的、不频繁的发布（通常以“季度”或“年”为单位）相比，敏捷方法大大提升了发布频率（通常以“天”或“周”为单位）。与传统的瀑布式开发模型相比，采用敏捷或迭代式开发意味着更频繁的发布、每次发布包含的变化更少



DevOps是**Development**和**Operations**的组词，通常看作敏捷开发的延续，重视软件开发人员（**Dev**）和IT运维技术人员（**Ops**）之间的沟通合作，将敏捷精神由开发阶段拓展到构建、测试、发布等过程，达到快速响应变化、交付价值的目的。尽管**DevOps**中没有保护**Test**，但开发者都不否认自动化测试是实现**DevOps**的重要基石之一



DevOps介绍



DevOps的引入能对产品交付、测试、功能开发和维护起到意义深远的影响。在缺乏DevOps能力的组织中，开发与运营之间常常存在信息“鸿沟”。运营人员要求更好的可靠性和安全性，开发人员则希望基础设施响应更快，而业务用户的需求则是更快地将更多的特性发布给最终用户使用。这种信息鸿沟就是最常出问题的地方



DevOps经常被描述为“开发团队与运营团队之间更具协作性、更高效的关系”。由于团队间协作关系的改善，整个组织的效率因此得到提升，伴随频繁变化而来的生产环境的风险也能得到降低

目录

CONTENTS

PART 01 开发者与软件测试

测试和调试

开发者测试

PIE模型

PART 02 开发者测试方法与技术

静态测试与动态测试

黑盒测试与白盒测试

PART 03 开发者测试趋势

软件开发困境

DevOps介绍

DevOps中开发者测试





DevOps中开发者测试



在当前环境下，基于敏捷的要求，企业要求的不仅仅是传统的测试人员进行测试，开发者首先需要对自己的代码负责，进行一定水准的测试。在开发者本地进行测试完成后，再将代码提交上服务器进行自动化的集成测试

baidu.com



在这个快速变化发展的时代，任何一款产品想要在市场具备竞争力，必须能够快速适应和应对变化，要求产品开发过程具备快速持续的高质量交付能力。而要做到快速持续的高质量交付，自动化测试将必不可少



DevOps中开发者测试



自动化测试工具云集，但做自动化也不要冲动，需要重视以下几点：

- ① 综合考虑项目技术栈和人员能力，采用合适的框架来实现自动化
- ② 结合测试金字塔和项目具体情况，考虑合适的测试分层，如果能够在底层测试覆盖的功能点一定不要放到上层的端到端测试来覆盖
- ③ 自动化测试用例设计需要考虑业务价值，尽量从用户真实使用的业务流程/业务场景来设计测试用例，让自动化优先覆盖到最关键的用户场景
- ④ 同等看待测试代码和开发代码，让其作为产品不可分割的一部分



DevOps中开发者测试



测试环境的准备在过去是一个比较麻烦和昂贵的事情，很多组织由于没有条件准备多个测试环境，导致测试只能在有限的环境进行，从而可能遗漏一些非常重要的缺陷，测试的成本和代价很高



随着云技术的发展，多个测试环境不再需要大量昂贵的硬件设备来支持，加上以Docker为典范的容器技术生态系统也在逐步成长和成熟，创建和复制测试环境变得简单多了，成本大大的降低



另一方面是大量开源工具的出现，这些工具往往都是轻量级的、简单易用，相对于那些重量级的昂贵的测试工具更容易被人们接受

谢谢欣赏